Detect, Localize, and Defend: Action-Centered Semantic Trajectory Graphs for the Agentic Web

Anonymous Author(s)

ABSTRACT

Language-model agents increasingly act on the Web through tools—searching, browsing, filling forms, and transacting on behalf of users—and every tool call can change external state. Existing screening either classifies flat text or scores an entire execution, leaving two questions unanswered: whether cross-step execution dependencies are represented at all, and which tool operation actually produced the risk. We reformulate agent risk analysis as joint graph-level anomaly detection and action-level subgraph localization over a semantic trajectory graph that encodes tasks, reasoning, tool calls, and environment states with typed, directed, and justified edges. We further propose Action execution units—inseparable semantic slices of intent, inputs, execution, and outcome—as the atomic unit of unit-wise candidate subgraphs, a contextual semantic-subgraph MIL model that aligns full trajectories and candidates through a shared edge-attention encoder and ranks risky actions under graph-level supervision with paired weak supervision, and safety experiences compiled from the top-ranked Action subgraph that condition later agent decisions, closing the loop from detection to defense; across TBD-EVALUATION-SCOPE, the proposed method achieves TBD-KEY-RESULTS under TBD-PROTOCOL.

CCS CONCEPTS

• **Security and privacy** → →; • **Computing methodologies** → →.

ACM Reference Format:

Anonymous Author(s). 2026. Detect, Localize, and Defend: Action-Centered Semantic Trajectory Graphs for the Agentic Web. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The Web is entering a phase in which a growing share of interactions is performed by software on behalf of people. Language-model agents interleave natural-language reasoning with tool calls to search, browse, fill forms, send messages, and transact [42, 58], and standardized tool interfaces such as the Model Context Protocol turn external services and data sources into first-class agent capabilities [17, 63]. Web-native benchmarks demonstrate agents operating realistic sites end to end [11, 69], tool-use suites exercise stateful, multi-step interactions [30, 34], API-based agents extend the reach of browsing [45], agentic search systems now return synthesized, citation-backed answers instead of link lists [15], and survey literature describes the resulting environment as an *agentic Web* in which autonomous, goal-driven agents are primary actors [39, 56, 62]. This shift reopens long-standing assumptions about who reads, writes, and pays for the Web, because the client is no longer a human reading a page but an agent executing operations—and every executed tool call can change external state.

The same shift changes the attack surface. Under indirect prompt injection, hostile instructions embedded in fetched content steer the model that processes them [16, 31, 32]; when the steered model is an agent, the instruction is executed through tools with real consequences [9, 61, 64]. Attacks and evaluations have followed this trajectory: environmental injection on generalist web agents [26], prompt injection against multimodal web agents [54], tool-layer attacks on protocol-mediated tool use [52], adaptive red-teaming of deployed web agents [47], and analyses showing that web agents are more vulnerable than standalone models precisely because each step multiplies exposure [7]. Attack success is commonly measured as an attack success rate over task runs [9, 61], and dedicated safety benchmarks confirm that autonomous web agents routinely complete tasks in policy-violating ways [25, 50]. Crucially for the Web, consequences are state changes—payments sent, records modified, private data exfiltrated—and recent work shows such consequences can be made covert, so the user never learns what their agent did [24].

Current defenses address this threat at the wrong granularity, and their collective blind spots can be stated as two unanswered questions: whether cross-step execution dependencies are represented at all, and which tool operation actually produced the risk. The first question exposes segment-level methods. Content-level hardening and detection operate on text spans or attention patterns [5, 6, 67], guard models classify messages against safety taxonomies [21, 29], and design-level systems constrain information flow between channels [8, 71]; all of them judge *segments* in isolation, while the risk of an agent run is a property of *compositions*: a benign query, a benign-looking tool output, and an otherwise reasonable plan can jointly produce an unauthorized transfer, and no per-segment verdict sees that propagation. The second question exposes run-level methods. Whole-trajectory judges score an entire execution [36, 59, 72] and runtime monitors predict or veto upcoming steps [27, 28, 35, 37, 53]; they answer *whether* a run went bad, but not *which* operation caused it, and outcome-only judges demonstrably miss process faults that do not change the final result [36]. An alert without a locus cannot become policy for the agent, cannot be audited by a site operator, and cannot be accumulated as knowledge; and since guardrails themselves are attackable and their verdicts are sensitive to formulation [70], evidence that is checkable and reusable has independent value.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

We argue that the root cause of both gaps is a mismatch between the unit of risk and the unit of execution. Risk in tool-using agents propagates along real execution dependencies: external state enters through a tool result, is adopted by reasoning, and materializes in a tool call whose output becomes the next context. The natural boundary of this propagation is the *action*: one tool call together with the state it consumed, the reasoning that justified it, and the state it produced. Reasoning has long been modeled as graphs in prompting frameworks [2, 57], and recent failure-attribution work shows that decisive errors can lie many steps away from their visible consequences, making dependency structure essential for locating them [10]. Yet no existing representation binds risk evidence to complete tool operations: fixed-k-hop neighborhoods cut through operations at arbitrary points, keeping half of a reasoning chain while discarding the state that motivated it.

This paper makes the Action execution unit the anchor of agent safety analysis. We represent each execution as a directed semantic trajectory graph whose nodes are problems, thoughts, actions, and states, and whose typed and directed edges carry justification text. We construct, for every action, an inseparable Action execution unit and expand candidate subgraphs by adding whole predecessor units along real dependencies, so that every candidate is a complete, connected slice of an actual execution. A contextual semantic-subgraph MIL model encodes the full trajectory and all candidates with a shared edge-attention encoder, fuses local scores with the global score, and is trained with graph-level labels plus paired weak supervision derived from normal/anomalous executions of the same task, whose changed actions identify likely loci. At inference, a validated threshold triggers candidate ranking, and the top-ranked Action subgraph is compiled by a generator—which never sees ground-truth labels—into a structured safety experience that conditions the agent's subsequent decisions, closing the loop from detection to defense. Our contributions are:

- a directed semantic trajectory graph representation for tool-using agents that jointly encodes tasks, reasoning, tool calls, and environment states, with the semantic type, direction, and justification of every edge;
- a candidate subgraph extraction and graph-level anomaly detection scheme in which candidate subgraphs are built on Action execution units—complete tool operations rather than fixed-hop neighborhoods—and a contextual semantic-subgraph MIL model that aligns full trajectories and candidates through a shared edge-attention encoder and, via learnable local–global aggregation and paired weak supervision, performs graph-level detection and Action-level localization jointly;
- a subgraph-level safety experience mechanism that compiles the top-ranked Action subgraph into a structured, node-grounded experience—generated without access to ground-truth labels—which conditions later agent decisions and closes the loop from detection to defense;
- an experimental design spanning detection, Action-level localization, and a controlled defense study whose conditions separate the effect of localized evidence from generic warning effects.

Section 8 develops why this formulation matters for the future Web: trajectory graphs as an auditable substrate and safety experiences as a portable safety layer. The rest of the paper formalizes the problem (§3), describes data and graph construction (§4), details the method (§5), specifies the evaluation design (§6–7), discusses implications for the agentic Web (§8), and concludes.

2 RELATED WORK

2.1 Attacks on Tool-Using and Web Agents

Research on language-model security began with adversarial inputs to a single model but has moved decisively toward tool-mediated, multi-step settings. Indirect prompt injection showed that content fetched by an application can steer the model that reads it [16, 32], and automated search over injection payloads generalizes the attack [31]. When the application is an agent, the injected instruction becomes executable: InjecAgent measures direct and indirect injections against tool-integrated agents [61], AgentDojo provides a dynamic environment with adaptive attackers [9], and the Agent Security Bench formalizes attack scenarios across agent scenarios [64]. Web-specific work shows that hostile page content can hijack browsing agents to leak private data [26] or manipulate multimodal action generation [54]; protocol-level studies extend the surface to tool discovery and selection under MCP [17, 52, 63]; and red-teaming frameworks adaptively search web agents for injection chains [47]. Security analyses attribute the elevated risk of web agents to multi-step processing of untrusted content [7], and safety benchmarks for autonomous web agents document policy-violating completions [25, 50]. Recent measurements emphasize a property directly relevant to detection: attacks can be made *covert*, leaving no user-visible trace of the compromised operation [24]. Our work takes this multi-step, state-changing threat model as given and asks what representation makes the resulting risk detectable and attributable.

2.2 Defenses: Content Hardening, Guards, Judges, and Runtime Monitors

Defenses cluster by the object they inspect. Content-level methods restructure queries or align preferences so instructions hidden in data are not followed [5, 6]; attention-based detectors localize anomalous attention mass over injected spans [67]; provable and design-level approaches enforce data-flow and capability constraints that make injections inert [8, 71]. Guard models classify inputs and outputs against safety taxonomies [21], with known calibration limitations [29]. Judge-style assessment scores whole interactions or trajectories [59, 66, 72], and analysis of outcome-only trajectory judges shows they miss faults that do not alter the final result, arguing for process-level evidence [36]. Runtime monitors predict risky next actions or reason over world models [27, 53], adapt lifelong [35], operate at step level [37], or produce diagnostic explanations [28]; guardrails themselves, however, can become denial-of-service targets and their verdicts are formulation-sensitive [70]. Across these families, outputs are either a scalar over text or a verdict over a run; the operation-level evidence needed to reconfigure an agent is rarely produced, and when it is (as in diagnostic frameworks) it is not coupled to a learned detector that can rank *which* operations are suspect. That coupling—joint detection,

localization, and conversion into agent-consumable knowledge—is the gap this paper addresses.

2.3 Graph Structure, Graph Anomaly Detection, and MIL

Modeling reasoning as a graph has precedent in prompting frameworks that branch and merge thoughts [2, 57], and dependency-guided attribution over execution graphs recently showed that decisive failures must be located with structural context [10]. On the modeling side, we build on attention-based and relational graph encoders [3, 19, 43, 51]. Graph anomaly detection offers detectors at node and graph levels [12, 33, 41, 48], including substructure-aware [12] and subgraph-aligned formulations [18], anomaly detection over edge-attributed multigraphs [46], and transferable graph-level scoring [55]. These methods, however, target social, transaction, or sensor graphs whose anomalies are unusual nodes or subgraph motifs; none instantiates a graph over agent executions, where the anomaly of interest is a *semantic* dependency pattern—an operation and its justification chain—rather than a structural outlier, and none provides operation-anchored localization. Multiple instance learning supplies the weak-supervision discipline we need: bag-level labels with instance-level ambiguity [4], attention-based aggregation [20], and graph-based bag encoders [49]. We instantiate bags as trajectories, instances as Action execution unit-anchored candidate subgraphs, and add paired supervision that classical MIL does not use.

2.4 Experience, Reflection, and the Closest Work

Agents already convert experience into reusable knowledge: verbal self-reflection improves retries [44], experience-driven learners extract and transfer insights across tasks [65], and recent systems industrialize reflective learning [1]; a unified review describes this trend as *externalization* of memory, skills, and protocols around the model [68]. This literature reuses experience for *competence*; we reuse it for *safety*, grounding each experience in a localized risk subgraph rather than a free-text lesson. The closest work to ours is DeFA [10], which attributes failures to decisive errors using dependency guidance over agent traces; DeFA is post-hoc—it explains failures after they are known—whereas our formulation detects anomalous trajectories *before* outcomes are trusted, localizes risk at the granularity of complete tool operations under only graph-level supervision, and additionally converts the localized evidence into safety experiences that condition future behavior. Runtime governance proposals argue for contracts and policies at execution time [13, 22, 38]; our detector and experience format are designed as the concrete substrate such contracts would audit.

3 PROBLEM FORMULATION

3.1 Tool-Using Agent Trajectories and Labels

A *trajectory* $\mathcal{T}$ records an agent run from receiving a task to producing a final result. It contains the user goal, plans and reflections, every tool invocation with its parameters, every environment-returned state, and the final result; implementations may interleave these records, and our representation only requires that each record carry its segment text and role. Each trajectory carries a graph label

$$y_G \in \{0, 1\}, \tag{1}$$

where $y_G = 0$ denotes a normal completion or a successful defense against an attempted attack, and $y_G = 1$ denotes that an attack produced a wrong, unauthorized, or leaking outcome. Data sources, labeling protocol, and quality control are reported as `TBD-DATA-PROVENANCE`, `TBD-LABELING-PROTOCOL`, and `TBD-DATA-QUALITY-CONTROL`.

Table 1: Node types of the semantic trajectory graph.

Node type	Meaning
`Problem`	user task or original goal
`Thinking`	agent plan, analysis, or reflection
`Action`	agent-initiated tool call
`State`	tool output, environment feedback, or final state

Table 2: Edge types of the semantic trajectory graph. Direction always follows execution flow.

Edge type	Meaning
`computes_from`	an Action produces a State
`depends_on`	later reasoning or operation uses earlier information
`sequential`	execution-order relation only
`reflects`	reasoning or conclusion reflects a state
`branches_from`	a plan branches from shared prior information

3.2 Semantic Trajectory Graph

We represent $\mathcal{T}$ as a directed attributed graph

$$G = (V, E, X_V, X_E), \tag{2}$$

where V is the node set, $E \subseteq V \times V$ the directed edge set, and X_V, X_E the node and edge features described in §4. Nodes carry one of four types reflecting execution roles (Table 1), and edges carry one of five relation types reflecting execution semantics (Table 2).

3.3 Action Execution Units

For an action $a \in V$, the Action execution unit is the inseparable semantic slice

$$U_a = \{a\} \cup R_a \cup I_a \cup O_a \cup F_a, \tag{3}$$

where R_a is the `Thinking` chain recovered along semantic in-edges that justified a; I_a is the set of `State`/`Problem` nodes feeding a or its reasoning; O_a is the `State` produced by a; and F_a is the set of reflections or results directly tied to O_a. The typical structure is

input State/Problem $\longrightarrow$ *Thinking chain* $\longrightarrow$ *Action* $\longrightarrow$ *output State,*

and the definition takes one complete tool operation as the minimal granularity rather than a node distance. This choice is deliberate: every candidate evaluated downstream corresponds to something the agent actually did, with the evidence that justified it attached.

3.4 Dependency-Aware Candidates

If the output state of unit U_{a_i} is consumed by unit U_{a_j}, we add a unit dependency $U_{a_i} \rightarrow U_{a_j}$. For an anchor unit, candidates are grown by adding complete predecessor units layer by layer:

$$C_a^{(0)} = U_a, \tag{4}$$

$$C_a^{(l+1)} = C_a^{(l)} \cup \bigcup_{U_p \in \mathrm{Pred}(C_a^{(l)})} U_p. \tag{5}$$

Candidates satisfy six constraints: each candidate is anchored at a real Action; all nodes and edges come from the original graph; each candidate is connected through real edges; each expansion adds complete execution units; independent branches form separate candidates while converging actions may form joint-context candidates; and the candidate budget may not remove every anchor candidate of any action. The candidate set of a trajectory is $C = \{C_k\}$.

3.5 Tasks

Given G with features and the candidate set C, the model must solve three coupled tasks. *Graph-level detection*: estimate $p_G = \Pr(y_G = 1 \mid G)$. *Action-level localization*: rank candidates $C_k \in C$ by $p_k = \Pr(C_k \text{ is a risk locus} \mid G)$, where the ground-truth locus for training purposes is derived only from weak signals (§5). *Defense*: convert the top-ranked localized evidence into a structured safety experience that changes agent behavior on future, similar situations, degrading attack success while preserving normal task utility. The three tasks share one mechanism chain: propagation structure motivates the graph; the execution boundary motivates the unit; and usable defense motivates keeping localization and evidence generation coupled to detection.

4 DATA AND SEMANTIC GRAPH CONSTRUCTION

4.1 Corpora

Our study draws on three abstract corpora, whose concrete identities, sizes, and distributions are reported in TBD-DATASET-NAME-A, TBD-DATASET-NAME-B, TBD-PRIVATE-DATASET-NAME, and TBD-DATASET-STATISTICS (Table 3). *Public Dataset A* consists of tool-integrated agent runs under injection scenarios, where the environment exposes tools and adversarial content enters through tool results; runs end in either a successful completion or a compromised outcome. *Public Dataset B* consists of web-agent interaction runs over realistic sites, providing the browsing-style executions that the semantic trajectory graph must also cover. *Private Dataset* is an in-house corpus of paired normal/anomalous executions of the same tasks, collected under the provenance and labeling protocol recorded in TBD-DATA-PROVENANCE and TBD-LABELING-PROTOCOL. Anomaly ratios, split protocol, grouping, and leakage controls are reported in TBD-ANOMALY-RATIO, TBD-SPLIT-PROTOCOL, and TBD-DATA-LEAKAGE-CONTROL; seeds for stochastic stages are reported in TBD-RANDOM-SEEDS. Trajectories that contain no `Action` node are excluded, since the formulation anchors all candidates at real tool operations.

Table 3: Corpus overview. All fields are completed by the authors from the underlying corpora.

Corpus	Source	Tasks	Label source	#Runs	Anomaly rate
TBD-DATASET-STATISTICS					

4.2 Labels and Matching Control

A trajectory is anomalous when the executed run ends in a wrong, unauthorized, or leaking outcome attributable to injected or otherwise hostile influence; a normal run either completes correctly or successfully resists an attempted attack. Labels are produced under the protocol in TBD-LABELING-PROTOCOL with quality control described in TBD-DATA-QUALITY-CONTROL. For the paired corpus, the matching control matters as much as the labels: a paired run pair must share the task, the agent configuration, and the tool inventory, differing only in the presence and placement of hostile influence, so that changed actions between the two runs identify candidate risk loci for weak supervision. Because benchmarks differ in what their success metrics capture, we record, for each source, exactly which ground-truth signal was available at which granularity.

4.3 Two-Phase Graph Construction

Construction proceeds in two phases with a strict ordering. *Phase 1 (execution skeleton)* recovers the basic loop from the trajectory records: `Thinking`→`Action`→`State` chains, `computes_from` edges from each action to its output state, `depends_on` edges from each action to the state or problem that triggered it, and `reflects` edges from reflections to the states they discuss. *Phase 2 (dependency expansion)* adds high-confidence cross-step dependencies, branch (`branches_from`) and convergence edges that Phase 1 could not determine from local records alone, using textual alignment between record contents. Two invariants hold throughout: Phase 2 may not modify nodes, node text, or edges fixed by Phase 1, and the construction pipeline never receives the graph label y_G, so no leakage path exists from labels into structure. Every edge additionally carries a *justification text* t_{ij}—the quotation or reference that supports why the dependency was added—which later feeds the edge encoder (§5) and makes each inferred dependency auditable.

4.4 Structural Audit

Because downstream localization depends on graph quality, we audit the constructed graphs before training: coverage of executed actions (every action must have at least one incoming dependency edge and one outgoing `computes_from` edge), candidate feasibility (every anchor must admit at least one candidate under the constraints of §3), and stability of unit boundaries across re-construction runs. Audit outcomes and any structural failure modes are reported in TBD-OBSERVED-LIMITATIONS. The same audit defines the candidate-coverage statistic reported alongside localization results, so that candidate construction quality is never confused with final localization accuracy.

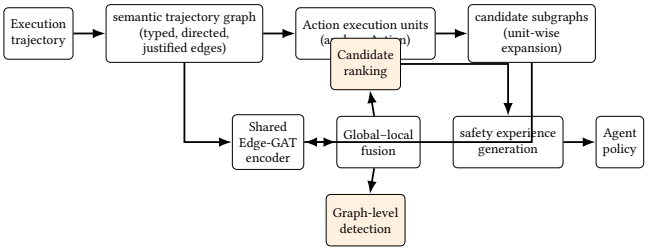


Figure 1: The overall architecture of the proposed framework. The upper part represents semantic trajectory graph construction and Action execution unit-anchored candidate subgraph extraction; the lower left part corresponds to shared edge-attention encoding and global–local fusion; and the right part illustrates graph-level detection, candidate ranking, and safety experience generation that conditions the agent policy.

5 METHODOLOGY

In this section, we present the proposed framework in detail. Unlike existing agent-safety methods that judge text segments or score entire executions, the proposed framework grounds every risk decision in complete tool operations: it extracts Action execution units from the semantic trajectory graph, grows candidate subgraphs along real execution dependencies, and scores each candidate against its full trajectory through one shared encoder. Moreover, instead of requiring action-level annotations, the framework learns from graph-level labels and paired normal/anomalous executions of the same task, and converts the top-ranked subgraph into a structured safety experience that changes agent behavior on future, similar situations. The framework is composed of three key components: *Semantic Trajectory Graph Representation and Action Execution Unit Construction*, *Contextual Semantic-Subgraph Encoding and Scoring*, and *Weakly Supervised Optimization with Safety Experience Generation*. The overall architecture is shown in Fig. 1.

5.1 Notations

We summarize the notation used throughout this section. Each execution trajectory $\mathcal{T}$ is represented as a directed attributed graph $G = (V, E, X_V, X_E)$ following Eq. (2) in §3, where V is the node set, $E \subseteq V \times V$ is the directed edge set following execution flow, X_V collects node features, and X_E collects edge features. Each node carries one of the four execution roles in Table 1, and each edge carries one of the five relation types in Table 2 together with a direction and a justification text t_{ij}. For an action $a \in V$, its Action execution unit is denoted U_a following Eq. (3), and the candidate subgraph set of the trajectory is $\mathcal{C} = \{C_1, \ldots, C_K\}$ with K candidates grown by Eqs. (4)–(5). A detection model is represented as f_θ, parameterized by θ; given G and $\mathcal{C}$, it outputs a graph-level score $p_G \in [0, 1]$ and candidate scores $\{p_k\}$ with margins $\{m_k\}$. Training minimizes a composite objective $\mathcal{L}_{\text{total}}(\theta)$ defined over graph labels y_G, paired normal/anomalous trajectories, and pseudo-labeled candidates. Unless otherwise specified, d denotes the hidden dimension, L the number of encoder layers, and B the maximum candidate size measured in nodes.

5.2 Chunk-Aware Node and Edge Representation

In this subsection, we construct the node and edge features that feed the shared encoder. Node texts in trajectories range from a tool parameter to a multi-paragraph reflection, and a single pooled vector would blur exactly the spans that carry risk signals. Specifically, each node text is split into overlapping token chunks $\{c_{ij}\}$, encoded independently by a sentence encoder [14, 40], and each chunk receives a risk gate:

$$z_{ij} = \text{TextEncoder}(c_{ij}), \qquad r_{ij} = \sigma(\text{MLP}(z_{ij})), \tag{6}$$

where $z_{ij} \in \mathbb{R}^d$ is the embedding of chunk j of node i, and $r_{ij} \in (0, 1)$ is the gate value that amplifies chunks whose content resembles risk-bearing language. The gated chunks are aggregated by max pooling into the node text representation:

$$h_i^{\text{text}} = \max_j \big(\phi(z_{ij}) \odot (1 + r_{ij})\big), \tag{7}$$

where $\phi(\cdot)$ is a linear projection that aligns chunk embeddings with the node feature space and $\odot$ denotes element-wise multiplication; taking the maximum over gates keeps the most risk-salient chunk of each node. The final node feature concatenates h_i^{text} with a node-type embedding, a relative position encoding, and the text length. Every directed edge (i, j) encodes its justification text, relation type, and direction:

$$g_{ij} = \text{MLP}_E\big(\big[\,\text{TextEncoder}(t_{ij});\, e_{\text{rel}(i,j)};\, e_{\text{dir}(i,j)}\,\big]\big), \tag{8}$$

where t_{ij} is the justification text of §4, e_{rel} and e_{dir} are learnable embeddings of the relation type in Table 2 and the execution direction, and $[\cdot\,;\cdot]$ denotes concatenation. This design allows every inferred dependency to remain auditable, since its feature is grounded in the quotation that supports it. Encoder identity, chunk length, overlap, and dimensions are reported in TBD-ENCODER-CONFIGURATION.

5.3 Action Execution Units and Candidate Subgraphs

In this subsection, we construct the Action execution units and the candidate subgraphs from the graph built in §4. For each action a, the unit U_a in Eq. (3) is recovered directly from the graph structure: the reasoning chain R_a follows semantic in-edges backward from a; the inputs I_a follow the same edges one step further or terminate at the triggering state; the output O_a follows the `computes_from` edge; and the reflections F_a attach through `reflects` edges. Figure 2 illustrates a unit inside a running example with branches and a convergence. Unit dependencies are then built from state reuse: if the output state of U_{a_i} is consumed by U_{a_j}, we add a unit dependency $U_{a_i} \rightarrow U_{a_j}$, and candidates grow from an anchor unit by adding complete predecessor units layer by layer, following Eqs. (4)–(5) under the six constraints of §3. Independent branches form separate candidates, so evidence from one branch is not diluted by another; converging actions may additionally form a joint-context candidate, so composition risks—where two individually innocuous operations jointly produce harm—are represented. The budget constraint guarantees that every action keeps at least one anchored candidate, so localization can never silently drop an operation. This construction ensures that every candidate is a complete, connected slice of an actual execution, which is the property

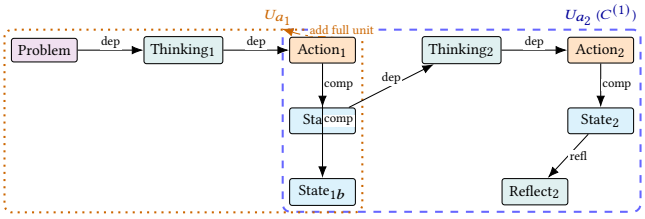


Figure 2: The construction of Action execution units and unit-wise candidate subgraphs. The boxed regions mark complete units: the anchored unit U_{a_2} contains the full input state, reasoning chain, action, output state, and reflection, and candidate growth adds complete predecessor units (U_{a_1}), never partial hops. Branch outputs of a_1 stay inside their own units and feed separate candidates.

that fixed-hop neighborhoods lack. Candidate coverage and size statistics are reported in TBD-CANDIDATE-COVERAGE.

5.4 Shared Edge-Attention Encoding and Contextual Scoring

In this subsection, we encode the full trajectory and all candidates with one shared encoder and score candidates in context. Full trajectories and candidate subgraphs must be comparable in a single representation space; otherwise the contextual comparison below degenerates. Specifically, a directed edge-attention layer aggregates from true in-edge neighbors only, with attention determined jointly by source, target, and edge representations:

$$h_i^{(l+1)} = \mathrm{ResNorm}\left(h_i^{(l)} + \sum_{j:(j,i)\in E} \alpha_{ji}^{(l)}\, \mathrm{Msg}(h_j^{(l)}, g_{ji})\right), \quad (9)$$

where $h_i^{(l)}$ is the state of node i at layer l, g_{ji} is the edge feature of Eq. (8), $\alpha_{ji}^{(l)}$ is the attention coefficient computed from the pair of node states and the edge representation, $\mathrm{Msg}(\cdot)$ is the message function, and $\mathrm{ResNorm}(\cdot)$ is a residual-and-normalization block that stabilizes depth. Because edge features enter both the attention logits and the message, justification text and relation type directly modulate information flow, the property that relational and heterogeneous encoders establish as necessary for typed graphs [19, 43]. Graph-level representations pool node states:

$$h_G = \mathrm{MeanPool}(\{h_i\}) \,\|\, \mathrm{MaxPool}(\{h_i\}), \quad (10)$$

where $\|$ denotes concatenation; Eq. (10) is computed identically for the full graph G and each candidate subgraph C_k, and both pass through a shared LayerNorm so that their scales are aligned.

For finer granularity, each candidate is scored *in context*: the same operation can be legitimate in one trajectory and fatal in another, so the candidate representation is compared against the full-graph representation through similarity, difference, and product terms:

$$q_k = \left[h_{C_k};\, h_G;\, |h_{C_k} - h_G|;\, h_{C_k} \odot h_G\right], \quad (11)$$

from which a candidate margin and probability are read:

$$m_k = f_\theta(q_k), \qquad p_k = \sigma(m_k), \quad (12)$$

where f_θ is a scoring head parameterized by θ, m_k is the candidate margin, and p_k is the probability that candidate C_k is a risk locus. The local branch fuses the strongest candidate with the average of the top-ranked candidates, so evidence scattered over several operations is not lost:

$$S_{\mathrm{local}} = \alpha \max_k m_k + (1-\alpha)\,\mathrm{Mean}(\mathrm{TopK}(\{m_k\})), \quad (13)$$

and the final graph-level score blends the full-graph and local branches:

$$S_G = \beta S_{\mathrm{full}} + (1-\beta) S_{\mathrm{local}}, \qquad p_G = \sigma(S_G), \quad (14)$$

where S_{full} is the score read from h_G by the same head, and $\alpha, \beta \in (0,1)$ are sigmoid-parameterized and learned from data rather than fixed, letting the training distribution decide how much localization contributes to detection. This design allows the local branch to contribute operation-level evidence that a whole-graph score cannot express, while the global branch supplies behavioral context that a subgraph alone lacks.

5.5 Weakly Supervised Optimization

In this subsection, we train the detector and the ranker jointly without action-level annotations. Complete action-level labels are scarce, while graph labels are abundant and paired normal/anomalous runs of the same task exist by construction of the corpora. Specifically, the total loss combines four terms:

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{graph}} + \lambda_p \mathcal{L}_{\mathrm{pseudo}} + \mathcal{U}(\mathcal{L}_{\mathrm{normal}}; s_n) + \mathcal{U}(\mathcal{L}_{\mathrm{rank}}; s_r), \quad (15)$$

where $\mathcal{L}_{\mathrm{graph}}$ is the graph-level cross-entropy on y_G; $\mathcal{L}_{\mathrm{pseudo}}$ builds candidate pseudo-labels from paired trajectories, marking the candidates anchored at actions that changed between the anomalous and the normal run of the same task as positive within their anomalous graph; $\mathcal{L}_{\mathrm{normal}}$ suppresses high candidate scores inside normal graphs, discouraging a trivial solution in which every candidate is suspicious; and $\mathcal{L}_{\mathrm{rank}}$ requires that, for a task with one normal and one anomalous run, the anomalous graph's local score exceeds the normal graph's. Because the three auxiliary terms have different noise levels, they are combined through bounded uncertainty weighting in the spirit of multi-task loss balancing [23]:

$$v = \mathrm{softplus}(s), \qquad w = \exp(-v) \in (0,1], \quad (16)$$

$$\mathcal{U}(\mathcal{L}; s) = w\,\mathcal{L} + v, \quad (17)$$

where s is the learnable uncertainty parameter of the corresponding auxiliary term. The bound keeps weights in $(0,1]$, so a noisy auxiliary term is down-weighted but never dominates by inflating its own uncertainty.

5.6 Safety Experience Generation

In this subsection, we convert the localized evidence into knowledge that changes agent behavior. Detection without a locus cannot be acted upon, and a locus without transferable knowledge is lost at the end of the run. Specifically, at inference, a threshold τ tuned on validation data gates the graph-level decision, and only flagged trajectories undergo candidate ranking. Among near-tied candidates the smaller candidate is preferred, and heavily overlapping candidates are deduplicated; the Top-1 candidate serves as the primary risk evidence and the remaining top-ranked candidates serve as auxiliary evidence for evaluation. The safety experience generator

receives only the detection scores, the anchor action, local node texts, the true semantic edges, and their justifications—*never* the ground-truth label—and emits a structured record:

`risk_type`, `what_went_wrong`, `warning_signals`, `recommended_agent_policy`, `supporting_node_ids`, `confidence`.

The record is grounded by construction: `supporting_node_ids` points into the candidate, so every claim can be checked against nodes and edges of the original graph. The experience is consumed as *soft safety context*, not as a hard block: before subsequent tool calls, the agent re-checks goal consistency, input trustworthiness, parameter and data-scope authorization, and potential disclosure or unauthorized consequence of the pending operation. This design separates the contribution of the detector (when to act) from that of the localized evidence (what to check), which the controlled conditions of §7 isolate experimentally. Qualitative cases are reported in TBD-QUALITATIVE-CASES.

5.7 Complexity Analysis

The computational complexity of the framework can be broken down by stage, each contributing differently based on the operations involved. Graph construction and candidate extraction are linear passes over the trajectory: recovering units and growing candidates visit each node and edge a constant number of times, giving $O(|V|+|E|)$ per trajectory. Text encoding dominates preprocessing: with n_c chunks per node on average and a transformer encoder whose self-attention costs $O(n_t^2)$ for a chunk of n_t tokens, encoding all nodes costs $O(|V|\, n_c\, n_t^2)$, and encoding edge justifications adds $O(|E|\, n_t^2)$. The shared Edge-GAT costs $O(L(|E|d+|V|d^2))$ per forward pass on the full graph; since each candidate contains at most B nodes and B edges, encoding all K candidates adds $O(K\,L\,(Bd+Bd^2))$, and contextual scoring adds $O(Kd)$ for the fusion heads. Experience generation is a single generation call over the Top-1 candidate, whose prompt is bounded by B nodes, so its cost is independent of the trajectory length. In summary, the training complexity is $O(|V|n_c n_t^2+|E|n_t^2+L(|E|d+|V|d^2)+KL(Bd+Bd^2))$, and inference adds only one candidate-ranking pass over flagged trajectories. For a fixed candidate budget $K\cdot B$, the framework exhibits linear growth in trajectory size, in the same range as standard GNN encoders, which supports auditing long executions.

6 EXPERIMENTAL SETUP AND EVALUATION DESIGN

6.1 Protocol

All experiments follow one protocol across corpora and tasks. Data are split under TBD-SPLIT-PROTOCOL, with grouping that prevents the same task family, site, or injected payload family from spanning training and test splits, and leakage controls audited as specified in TBD-DATA-LEAKAGE-CONTROL. Stochastic stages are repeated over TBD-RANDOM-SEEDS seeds. The anomaly class is expected to be the minority in every corpus; therefore *PR-AUC* and anomaly-class *precision/recall/F1* are treated as the primary detection statistics, with accuracy reported only as a reference point, since accuracy can be dominated by the majority class under imbalance. Sensitivity of operating points is analyzed separately from ranking quality in TBD-THRESHOLD-ANALYSIS.

Algorithm 1 Training, Inference, and Deployment Procedure of the Proposed Framework

Require: trajectory corpus with graph labels $\{(\mathcal{T}_i, y_{G,i})\}$; paired normal/anomalous executions $\mathcal{P}$; validation-tuned threshold τ; hyper-parameters.
Ensure: graph-level scores $\{p_G\}$; ranked candidates $\{p_k\}$; safety experience store $\mathcal{M}$.

/ Stage 1: graph construction and candidate extraction (offline, §4) */*
for each trajectory $\mathcal{T}$ **do**
  build the semantic trajectory graph G: Phase 1 fixes the execution skeleton, Phase 2 adds cross-step dependencies with justification texts t_{ij} (§4);
  recover Action execution units by Eq. (3) and grow candidate subgraphs by Eqs. (4)–(5) under the six constraints of §3;
end for
/ Stage 2: representation, encoding, and weakly supervised optimization */*
for each training epoch **do**
  for each mini-batch of graphs and candidate sets (G, C) **do**
    encode chunk-gated node features and edge features by Eqs. (6)–(8);
    encode G and all candidates with the shared Edge-GAT by Eqs. (9)–(10);
    compute candidate margins and graph-level scores by Eqs. (11)–(14);
    build paired pseudo-labels from $\mathcal{P}$; optimize $\mathcal{L}_{\text{total}}$ in Eq. (15) with bounded weights in Eqs. (16)–(17);
  end for
end for
/ Stage 3: inference and experience generation */*
for each run with $p_G \geq \tau$ **do**
  rank candidates by p_k; deduplicate overlapping candidates and prefer the smaller near-tied one;
  generate the safety experience from the Top-1 candidate: risk type, failure description, warning signals, recommended policy, supporting node ids, and confidence (§5.6);
  append the experience to the store $\mathcal{M}$;
end for
/ Stage 4: deployment loop with experience-conditioned execution */*
while the agent executes a new task **do**
  before each tool call, apply the soft-context re-checks induced by $\mathcal{M}$: goal consistency, input trustworthiness, parameter and data-scope authorization, and disclosure risk (§5.6);
  execute the tool call and extend the semantic trajectory graph with the observed records;
  if the run is flagged ($p_G \geq \tau$) **then**
    compile the localized evidence into a new safety experience and append it to $\mathcal{M}$;
  end if
end while
return graph-level decisions, ranked candidates, and the safety experience store $\mathcal{M}$.

Table 4: Main graph-level detection comparison. Groups are separated by input paradigm; cells are completed from the experiments.

Method	ROC-AUC	PR-AUC	F_1^{anom}	FPR	Bal-Acc
Text-trajectory screening					
Generic GNN					
Relation/edge-aware GNN					
Graph anomaly detection					
LLM-as-a-Judge					
Guard model					
Ours (full)					
TBD-MAIN-DETECTION-TABLE					

6.2 Baselines and Input Paradigms

Baselines cover six families, spanning the input paradigms that matter for interpretation (Table 4). (i) *Text-trajectory screening*: safety models and classifiers that read the trajectory as flat text or segmented text, representative of guard-model practice [21, 29]. (ii) *Generic graph neural networks*: message passing without relation or edge awareness [51]. (iii) *Relation- and edge-aware graph models*: encoders that consume typed, directed, attributed graphs [3, 19, 43]. (iv) *Graph anomaly detection*: graph-level scoring methods [33, 48, 55]. (v) *LLM-as-a-Judge*: generative judges over whole trajectories [59, 66, 72]. (vi) *Content-safety guard models*: deployed moderation systems applied to serialized runs. The final baseline roster, configurations, and the fairness protocol are recorded in TBD-BASELINE-LIST and TBD-BASELINE-CONFIGURATIONS. One reporting rule is fixed in advance: methods that read flat text and methods that read the semantic trajectory graph are *not* presented as operating under identical input conditions, and comparison tables group them accordingly (TBD-FAIR-COMPARISON-PROTOCOL).

6.3 Detection Questions

RQ1. Does joint global–local modeling improve graph-level detection over single-granularity scoring? The full model is compared against full-graph-only and Action-subgraph-only variants under the global/local ablation TBD-GLOBAL-LOCAL-ABLATION; the outcome, its significance, and the threshold analysis are recorded in TBD-DETECTION-SIGNIFICANCE and TBD-THRESHOLD-ANALYSIS. The mechanism claim under test is that the local branch adds operation-level evidence that a whole-graph score cannot express, while the global branch supplies behavioral context that a subgraph alone lacks; the ablation is designed so that each branch's marginal value is directly readable from the table.

6.4 Localization Questions

RQ2. Does localization recover the responsible operations? Metrics are Action Hit@1, Hit@K/Recall@K, node- and edge-level IoU between the predicted candidate and the ground-truth changed-action context, and candidate size. Candidate-generation coverage is reported separately in TBD-CANDIDATE-COVERAGE and never substituted for final localization: a candidate set that contains the true locus does not imply the model ranks it first. Results appear in TBD-LOCALIZATION-RESULTS; qualitative cases are curated in TBD-QUALITATIVE-CASES.

Table 5: Action-level localization. Coverage of the candidate stage is reported separately from final ranking.

Candidate scheme	Hit@1	Hit@K	IoU_n	IoU_e	Size
Fixed-hop candidates					
Action-unit candidates (ours)					
TBD-LOCALIZATION-TABLE					

Table 6: Representation and objective ablations.

Variant	ROC-AUC	PR-AUC	Hit@1
Representation ablations			
Objective ablations			
TBD-ABLATION-TABLE			

RQ3. Is the Action execution unit the right atomic unit? The central comparison is fixed-hop candidate construction versus unit-wise construction, holding the encoder and objective fixed, on coverage, final Hit@K, IoU, and candidate size (TBD-FIXED-HOP-VS-ACTION-UNIT). This isolates the representation choice from the scoring machinery: any difference must come from where candidate boundaries fall, not from parameters.

6.5 Ablation Questions

RQ4. Which representation and objective components carry the effect? The planned grid removes: edge justification text; relation type; directionality; replaces chunk-risk-max pooling with truncation or chunk-mean; fixes versus learns α, β; removes the shared LayerNorm alignment; removes paired pseudo-labels; and replaces bounded uncertainty weighting with fixed auxiliary weights. Results are recorded in TBD-REPRESENTATION-ABLATIONS and TBD-OBJECTIVE-ABLATIONS (Table 6).

RQ5. Does the pipeline transfer across input paradigms and corpora? The same protocol is run on TBD-DATASET-NAME-A, TBD-DATASET-NAME-B, and TBD-PRIVATE-DATASET-NAME with per-corpus statistics in TBD-DATASET-STATISTICS; cross-corpus variation is where representation choices are most likely to matter, since browsing-style runs and tool-benchmark runs differ in graph shape.

7 EXPERIENCE-BASED DEFENSE EVALUATION

7.1 Closed-Loop Protocol

The defense experiment measures whether localized evidence changes agent behavior on future, similar situations, and—critically—which part of the pipeline produces the change. The protocol instantiates the agent under the target attack suite, injects safety experiences as *soft safety context* before each tool call, and compares attack success and normal utility across conditions. All conditions share the agent configuration, the tool inventory, and the task distribution; the *only* manipulated variable is the content

Table 7: Defense conditions and outcomes. Conditions differ only in the safety-context content; numbers are completed from the experiments.

Condition	ASR↓	Task SR	False trig.	Cost	ΔASR
Attack-only baseline				–	–
Generic warning					
Full-graph experience					
Random subgraph experience					
Detected Action-subgraph exp. (ours)					
Oracle Action-subgraph exp.					

TBD-DEFENSE-TABLE

of the safety context. Attack success rate and task success rate follow the definitions of the underlying benchmarks [9, 25, 50, 61], and we additionally track *false triggers* (defensive interventions on benign operations) and intervention cost. Statistical comparisons between paired conditions use TBD-DEFENSE-SIGNIFICANCE. Because adversaries adapt [47, 60], reported suppression reflects the studied attack suite rather than a permanent property, which the deployment discussion in §8 takes up. The full defense table and its results are recorded in TBD-DEFENSE-TABLE (Table 7).

7.2 Controlled Conditions

Six conditions separate confounds that generic warnings cannot. *Attack-only* calibrates the undefended success rate. *Generic warning* injects a fixed alert that an attack may be present, with no localized evidence; it captures placebo effects of "be careful" prompting and is the condition most defenses implicitly implement. *Full-graph experience* injects the generator's output over the *entire* trajectory, testing whether the benefit comes from localization or merely from more grounded text. *Random subgraph experience* injects an experience grounded in a random connected candidate subgraph, controlling for grounding structure without risk relevance. *Detected Action-subgraph experience* is the full pipeline: the detector flags the run, the ranker selects the candidate, and the generator compiles the experience. *Oracle Action-subgraph experience* grounds the experience in the true risk locus, bounding what perfect localization could achieve. The gap between the detected and oracle conditions is therefore an upper-bound estimate of the localization component's headroom, and the gap between detected and generic conditions isolates the value of operation-level evidence over generic caution.

7.3 What Each Comparison Answers

Three contrasts carry the interpretation. Detected versus *generic* answers whether structured, operation-anchored evidence matters beyond alerting the agent that something might be wrong. Detected versus *full-graph* answers whether localization, rather than additional grounded context, is the operative ingredient. Detected versus *random subgraph* answers whether the grounding subgraph must be risk-relevant, not merely well-formed and connected. Utility is evaluated on normal runs, where a well-calibrated system must not intervene: false-trigger rate and utility degradation on the normal set are reported alongside attack suppression, since a defense that refuses benign operations fails the deployment test regardless of its attack success reduction. Cross-configuration generalization—experiences generated from one agent configuration consumed by another—is reported in TBD-DEFENSE-TABLE as an explicit column, because safety experiences are intended as reusable artifacts rather than session-bound outputs. Qualitative examples of generated experiences and induced policy changes appear in TBD-QUALITATIVE-CASES.

8 IMPLICATIONS FOR THE AGENTIC WEB

The Brave New Ideas track asks what the Web could become if a capability existed that does not exist today. We argue that action-centered execution auditing is that kind of capability, and we sketch what changes if it works at Web scale.

From content auditing to execution auditing. The Web's existing safety infrastructure—spam filters, content moderation, phishing detection, provenance signals—inspects *content* at fetch time. For agents, the dangerous event is not the fetched byte but the executed operation, and attacks can be invisible in content and outcome alike [24]. If browsers, agent runtimes, and tool protocols emit trajectory graphs as a first-class artifact—the same way access logs and network traces are emitted today—then every agent-mediated action carries an auditable record of what state motivated it, what reasoning justified it, and what it changed. Detection and localization over such records generalize beyond prompt injection: any policy violation, error, or fraud that manifests in execution structure becomes addressable with the same substrate, including failures that outcome-based evaluation misses [10, 36]. This is a concrete, engineering-shaped path from today's ad-hoc guardrails toward execution-layer trust for the agentic Web [38, 56].

Experiences as a portable safety layer. The safety experience format is deliberately an artifact, not a log line: it is small, structured, grounded in cited node identifiers, and written for a consumer agent rather than a human analyst. If experiences accumulate across runs, sites, and agent configurations, safety knowledge stops being bound to a single deployment—the failure a search agent suffers today can protect a booking agent tomorrow, and the memory-and-skill externalization trend of the field [1, 65, 68] extends from competence to safety. The defense conditions of §7 are designed to measure exactly the property this vision needs: whether an experience grounded in localized evidence transfers beyond the session that produced it.

Open problems the substrate raises. An execution-audit layer creates problems we do not solve here. *Standardization*: trajectory-graph emission needs schema-level agreement among runtimes, in the spirit of emerging tool-protocol standards [17, 63]. *Privacy*: graphs record how agents act on behalf of users, so emission policies must decide what is stored, for how long, and under whose access control. *Experience pollution*: shared experience pools can be poisoned, and an attacker who understands the format can craft content that generates misleading experiences; provenance and robustness of the experience channel are prerequisites for the ecosystem, echoing lessons from repository- and knowledge-poisoning research. *Governance*: if audits attribute actions to specific agent configurations, the substrate becomes evidence in accountability disputes, which is a feature for site operators and a liability question for agent vendors. We state these as open problems because they require the community-level agreement that a research paper

cannot produce; what this paper contributes is the demonstration that the substrate's core operation—joint detection, operation-level localization, and grounded experience generation—is learnable at all.

9 CONCLUSION

We reformulated the safety of tool-using Web agents as joint graph-level detection and operation-level localization over execution-grounded semantic trajectory graphs, anchored risk at complete Action execution units instead of fixed-hop neighborhoods, learned both tasks jointly through a contextual semantic-subgraph MIL model with shared edge-attention encoding, graph labels, and paired weak supervision, and closed the loop by compiling localized evidence into structured safety experiences that condition future agent behavior. The controlled defense design separates the effect of localized evidence from generic alerting, and the formulation gives the emerging agentic Web a substrate—trajectory graphs as first-class artifacts—on which execution auditing, cross-agent experience sharing, and governance can build. The approach's power is bounded by graph-construction coverage and by the faithfulness of paired weak supervision, and establishing robustness to adaptive adversaries and experience pollution remains future work; what this paper establishes is that detection, localization, and defense over execution structure is a coherent, learnable, and Web-relevant research direction.

REFERENCES

[1] Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. 2026. Experiential Reflective Learning for Self-Improving LLM Agents. arXiv:2603.24639. arXiv:2603.24639 http://arxiv.org/abs/2603.24639v2

[2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17682–17690. https://doi.org/10.1609/aaai.v38i16.29720 arXiv:2308.09687

[3] Shaked Brody, Uri Alon, and Eran Yahav. 2021. How Attentive are Graph Attention Networks? arXiv:2105.14491. arXiv:2105.14491 http://arxiv.org/abs/2105.14491v3

[4] Marc-André Carbonneau, Veronika Cheplygina, Eric Granger, and Ghyslain Gagnon. 2016. Multiple Instance Learning: A Survey of Problem Characteristics and Applications. https://doi.org/10.1016/j.patcog.2017.10.009 arXiv:1612.03365

[5] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363. arXiv:2402.06363 http://arxiv.org/abs/2402.06363v2

[6] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2833–2847. https://doi.org/10.1145/3719027.3744836 arXiv:2410.05451

[7] Jeffrey Yang Fan Chiang, Seungjae Lee, Jia-Bin Huang, Furong Huang, and Yizheng Chen. 2025. Why Are Web AI Agents More Vulnerable Than Standalone LLMs? A Security Analysis. arXiv:2502.20383. arXiv:2502.20383 http://arxiv.org/abs/2502.20383v3

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. arXiv:2503.18813 http://arxiv.org/abs/2503.18813v2

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37*. 82895–82920. https://doi.org/10.52202/079017-2636 arXiv:2406.13352

[10] Bo Deng, Xinlei Zheng, Yi Wei, Kang Zhou, Chongyang Tao, Renzhao Liang, Xuanren Chen, Lifan Guo, and Chi Zhang. 2026. DeFA: Dependency-Guided Failure Attribution for LLM Agents. arXiv:2610.01256. arXiv:2610.01256 http://arxiv.org/abs/2610.01256v1

[11] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36*. 28091–28114. https://doi.org/10.52202/075280-1220 arXiv:2306.06070

[12] Jingcan Duan, Bin Xiao, Siwei Wang, Haifang Zhou, and Xinwang Liu. 2024. ARISE: Graph Anomaly Detection on Attributed Networks via Substructure Awareness. *IEEE Transactions on Neural Networks and Learning Systems* (2024), 18172–18185. https://doi.org/10.1109/TNNLS.2023.3312655 arXiv:2211.15255

[13] Herman Errico, Jiquan Ngiam, and Shanita Sojan. 2025. Securing the Model Context Protocol (MCP): Risks, Controls, and Governance. arXiv:2511.20920. arXiv:2511.20920 http://arxiv.org/abs/2511.20920v1

[14] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 6894–6910. https://doi.org/10.18653/v1/2021.emnlp-main.552 arXiv:2104.08821

[15] Boyu Gou, Zanming Huang, Yuting Ning, Yu Gu, Michael Lin, Weijian Qi, Andrei Kopanev, Botao Yu, Bernal Jiménez Gutiérrez, Yiheng Shu, Chan Hee Song, Jiaman Wu, Shijie Chen, Hanane Nour Moussa, Tianshu Zhang, Jian Xie, Yifei Li, Tianci Xue, Zeyi Liao, Kai Zhang, Boyuan Zheng, Zhaowei Cai, Viktor Rozgic, Morteza Ziyadi, Huan Sun, and Yu Su. 2025. Mind2Web 2: Evaluating Agentic Search with Agent-as-a-Judge. arXiv:2506.21506. arXiv:2506.21506 http://arxiv.org/abs/2506.21506v2

[16] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 79–90. https://doi.org/10.1145/3605764.3623985 arXiv:2302.12173

[17] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278. arXiv:2503.23278 http://arxiv.org/abs/2503.23278v3

[18] Jingtao Hu, Bin Xiao, Hu Jin, Jingcan Duan, Siwei Wang, Zhao Lv, Siqi Wang, Xinwang Liu, and En Zhu. 2025. SAMCL: Subgraph-Aligned Multiview Contrastive Learning for Graph Anomaly Detection. *IEEE Transactions on Neural Networks and Learning Systems* (2025). https://doi.org/10.1109/TNNLS.2023.3323274

[19] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous Graph Transformer. In *Proceedings of The Web Conference 2020*. 2704–2710. https://doi.org/10.1145/3366423.3380027 arXiv:2003.01332

[20] Maximilian Ilse, Jakub M. Tomczak, and Max Welling. 2018. Attention-based Deep Multiple Instance Learning. arXiv:1802.04712. arXiv:1802.04712 http://arxiv.org/abs/1802.04712v4

[21] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674. arXiv:2312.06674 http://arxiv.org/abs/2312.06674v1

[22] Anupam Joshi, Tim Finin, Karuna Pande Joshi, and Lalana Kagal. 2026. Deontic Policies for Runtime Governance of Agentic AI Systems. In *2026 IEEE International Conference on Web Services (ICWS), Sydney, Australia, 2026, pp. 1255-1264,*. https://doi.org/10.1109/ICWS72778.2026.00159 arXiv:2606.19464

[23] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7482–7491. https://doi.org/10.1109/CVPR.2018.00781 arXiv:1705.07115

[24] Yunseok Lee, Yunji Kim, and Woojin Lee. 2026. Will the User Ever Know? Covert Indirect Prompt Injection Attacks on Tool-Using LLM Agents. arXiv:2608.30362. arXiv:2608.30362 http://arxiv.org/abs/2608.30362v3

[25] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, Nir Mashkif, and Segev Shlomov. 2024. ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents. arXiv:2410.06703. arXiv:2410.06703 http://arxiv.org/abs/2410.06703v7

[26] Zeyi Liao, Lingbo Mo, Chejian Xu, Mintong Kang, Jiawei Zhang, Chaowei Xiao, Yuan Tian, Bo Li, and Huan Sun. 2024. EIA: Environmental Injection Attack on Generalist Web Agents for Privacy Leakage. arXiv:2409.11295. arXiv:2409.11295 http://arxiv.org/abs/2409.11295v5

[27] Wenhao Lin, Chenyu Yu, Xingwei Lin, Sicong Cao, Xiang Chen, Lei Xue, Le Yu, Letian Sha, and Chunming Wu. 2026. DreamGuard: Efficient Runtime Guardrail for LLM Agents via Risk-Aware World Model. arXiv:2608.05695. arXiv:2608.05695 http://arxiv.org/abs/2608.05695v1

[28] Dongrui Liu, Qihan Ren, Chen Qian, Shuai Shao, Yuejin Xie, Yu Li, Zhonghao Yang, Haoyu Luo, Peng Wang, Qingyu Liu, Binxin Hu, Ling Tang, Jilin Mei, Dadi Guo, Leitao Yuan, Junyao Yang, Guanxu Chen, Qihao Lin, Yi Yu, Bo Zhang, Jiaxuan Guo, Jie Zhang, Wenqi Shao, Huiqi Deng, Zhiheng Xi, Wenjie Wang, Wenxuan Wang, Wen Shen, Zhikai Chen, Haoyu Xie, Jialing Tao, Juntao Dai, Jiaming Ji, Zhongjie Ba, Linfeng Zhang, Yong Liu, Quanshi Zhang, Lei Zhu, Zhihua Wei, Hui Xue, Chaochao Lu, Jing Shao, and Xia Hu. 2026. AgentDoG: A Diagnostic Guardrail Framework for AI Agent Safety and Security. arXiv:2601.18491.

arXiv:2601.18491 http://arxiv.org/abs/2601.18491v2

[29] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. 2024. On Calibration of LLM-based Guard Models for Reliable Content Moderation. arXiv:2410.10414. arXiv:2410.10414 http://arxiv.org/abs/2410.10414v2

[30] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2023. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688. arXiv:2308.03688 http://arxiv.org/abs/2308.03688v3

[31] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and Universal Prompt Injection Attacks against Large Language Models. arXiv:2403.04957. arXiv:2403.04957 http://arxiv.org/abs/2403.04957v1

[32] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. 2023. Prompt Injection attack against LLM-integrated Applications. arXiv:2306.05499. arXiv:2306.05499 http://arxiv.org/abs/2306.05499v3

[33] Yixin Liu, Zhao Li, Shirui Pan, Chen Gong, Chuan Zhou, and George Karypis. 2021. Anomaly Detection on Attributed Networks via Contrastive Self-Supervised Learning. *IEEE Transactions on Neural Networks and Learning Systems* (2021). https://doi.org/10.1109/TNNLS.2021.3068344 arXiv:2103.00113

[34] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2024. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682. arXiv:2408.04682 http://arxiv.org/abs/2408.04682v2

[35] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. arXiv:2502.11448. arXiv:2502.11448 http://arxiv.org/abs/2502.11448v2

[36] Hadi Mohammadi. 2026. trajectory-judge: What Outcome-Only LLM Judges Miss on Agent Trajectories. arXiv:2609.00038. arXiv:2609.00038 http://arxiv.org/abs/2609.00038v2

[37] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. arXiv:2601.10156. arXiv:2601.10156 http://arxiv.org/abs/2601.10156v1

[38] Albus W. Ng, Yi Han, Jusheng Zhang, and Wenhao Wang. 2026. Agent Safety Should Be a Runtime Contract. arXiv:2608.11274. arXiv:2608.11274 http://arxiv.org/abs/2608.11274v1

[39] Xiaohang Nie, Zihan Guo, Kezhuo Yang, Zhichong Zheng, Bochen Ge, Shuai Pan, Zeyi Chen, Youling Xiang, Yu Zhang, Weiwen Liu, Yuanjian Zhou, and Weinan Zhang. 2026. Synergy: A Next-Generation General-Purpose Agent for Open Agentic Web. arXiv:2603.28428. arXiv:2603.28428 http://arxiv.org/abs/2603.28428v1

[40] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 3980–3990. https://doi.org/10.18653/v1/D19-1410 arXiv:1908.10084

[41] Amit Roy, Juan Shu, Jia Li, Carl Yang, Olivier Elshocht, Jeroen Smeets, and Pan Li. 2024. GAD-NR: Graph Anomaly Detection via Neighborhood Reconstruction. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 576–585. https://doi.org/10.1145/3616855.3635767

[42] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36*. 68539–68551. https://doi.org/10.52202/075280-2997 arXiv:2302.04761

[43] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *The Semantic Web: Research and Applications (Lecture Notes in Computer Science)*. 593–607. https://doi.org/10.1007/978-3-319-93417-4_38 arXiv:1703.06103

[44] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36*. 8634–8652. https://doi.org/10.52202/075280-0377 arXiv:2303.11366

[45] Yueqi Song, Frank Xu, Shuyan Zhou, and Graham Neubig. 2024. Beyond Browsing: API-Based Web Agents. arXiv:2410.16464. arXiv:2410.16464 http://arxiv.org/abs/2410.16464v3

[46] Konstantinos Sotiropoulos, Lingxiao Zhao, Pierre Jinghong Liang, and Leman Akoglu. 2023. ADAMM: Anomaly Detection of Attributed Multi-graphs with Metadata: A Unified Neural Network Approach. arXiv:2311.07355. arXiv:2311.07355 http://arxiv.org/abs/2311.07355v2

[47] Georgios Syros, Evan Rose, Brian Grinstead, Christoph Kerschbaumer, William Robertson, Cristina Nita-Rotaru, and Alina Oprea. 2026. MUZZLE: Adaptive Agentic Red-Teaming of Web Agents Against Indirect Prompt Injection Attacks. arXiv:2602.09222. arXiv:2602.09222 http://arxiv.org/abs/2602.09222v2

[48] Jianheng Tang, Jiajin Li, Ziqi Gao, and Jia Li. 2022. Rethinking Graph Neural Networks for Anomaly Detection. arXiv:2205.15508. arXiv:2205.15508 http://arxiv.org/abs/2205.15508v1

[49] Ming Tu, Jing Huang, Xiaodong He, and Bowen Zhou. 2019. Multiple instance learning with graph neural networks. arXiv:1906.04881. arXiv:1906.04881 http://arxiv.org/abs/1906.04881v1

[50] Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. 2025. SafeArena: Evaluating the Safety of Autonomous Web Agents. arXiv:2503.04957. arXiv:2503.04957 http://arxiv.org/abs/2503.04957v1

[51] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. Graph Attention Networks. arXiv:1710.10903. arXiv:1710.10903 http://arxiv.org/abs/1710.10903v3

[52] Che Wang, Jiaming Zhang, Ziqi Zhang, Zijie Wang, Yinghui Wang, Jianbo Gao, Tao Wei, Zhong Chen, and Wei Yang Bryan Lim. 2026. AdapTools: Adaptive Tool-based Indirect Prompt Injection Attacks on Agentic LLMs. arXiv:2602.20720. arXiv:2602.20720 http://arxiv.org/abs/2602.20720v1

[53] Haoyu Wang, Christopher M. Poskitt, Jiali Wei, and Jun Sun. 2025. ProbGuard: Proactive Runtime Monitoring for LLM Agent Safety via Probabilistic Prediction. In *Proc. ASE'26, pages 1470-1482. ACM, 2026.* https://doi.org/10.1145/3832783.3837413 arXiv:2508.00500

[54] Xilong Wang, John Bloch, Zedian Shao, Yuepeng Hu, Shuyan Zhou, and Neil Zhenqiang Gong. 2025. WebInject: Prompt Injection Attack to Web Agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2010–2030. https://doi.org/10.18653/v1/2025.emnlp-main.104 arXiv:2505.11717

[55] Xudong Wang, Chris Ding, Tongxin Li, and Jicong Fan. 2026. GLASS: Graph-Language Alignment with Spherical Scoring for Transferable Graph-Level Anomaly Detection. arXiv:2609.05253. arXiv:2609.05253 http://arxiv.org/abs/2609.05253v2

[56] Yingxuan Yang, Mulei Ma, Yuxuan Huang, Huacan Chai, Chenyu Gong, Haoran Geng, Yuanjian Zhou, Ying Wen, Meng Fang, Muhao Chen, Shangding Gu, Ming Jin, Costas Spanos, Yang Yang, Pieter Abbeel, Dawn Song, Weinan Zhang, and Jun Wang. 2025. Agentic Web: Weaving the Next Web with AI Agents. arXiv:2507.21206. arXiv:2507.21206 http://arxiv.org/abs/2507.21206v1

[57] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems 36*. 11809–11822. https://doi.org/10.52202/075280-0517 arXiv:2305.10601

[58] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. arXiv:2210.03629 http://arxiv.org/abs/2210.03629v3

[59] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 1467–1490. https://doi.org/10.18653/v1/2024.findings-emnlp.79 arXiv:2401.10019

[60] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. 2025. Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 7101–7117. https://doi.org/10.18653/v1/2025.findings-naacl.395 arXiv:2503.00061

[61] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. https://doi.org/10.18653/v1/2024.findings-acl.624 arXiv:2403.02691

[62] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2024. Large Language Model-Brained GUI Agents: A Survey. arXiv:2411.18279. arXiv:2411.18279 http://arxiv.org/abs/2411.18279v12

[63] Dongsen Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. 2025. MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents. arXiv:2510.15994. arXiv:2510.15994 http://arxiv.org/abs/2510.15994v2

[64] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2024. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. arXiv:2410.02644. arXiv:2410.02644 http://arxiv.org/abs/2410.02644v4

[65] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642. https://doi.org/10.1609/aaai.v38i17.29936 arXiv:2308.10144

[66] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36*. 46595–46623. https://doi.org/10.52202/075280-2020 arXiv:2306.05685

[67] Yinan Zhong, Qianhao Miao, Yanjiao Chen, Jiangyi Deng, Yushi Cheng, and Wenyuan Xu. 2026. Attention is All You Need to Defend Against Indirect Prompt Injection Attacks in LLMs. In *Proceedings of the 2026 Network and Distributed System Security Symposium*. https://doi.org/10.14722/ndss.2026.240394 arXiv:2512.08417

[68] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. 2026. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. arXiv:2604.08224. arXiv:2604.08224 http://arxiv.org/abs/2604.08224v1

[69] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854. arXiv:2307.13854 http://arxiv.org/abs/2307.13854v4

[70] Yuguang Zhou, Xunguang Wang, Pingchuan Ma, Zhantong Xue, Zhaoyu Wang, and Shuai Wang. 2026. From Shield to Target: Denial-of-Service Attacks on LLM-Based Agent Guardrails. arXiv:2606.14517. arXiv:2606.14517 http://arxiv.org/abs/2606.14517v2

[71] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. 2025. MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. arXiv:2502.05174. arXiv:2502.05174 http://arxiv.org/abs/2502.05174v4

[72] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2024. Agent-as-a-Judge: Evaluate Agents with Agents. arXiv:2410.10934. arXiv:2410.10934 http://arxiv.org/abs/2410.10934v2